Day 3 — Deep Learning for Time-Series Anomalies

From leakage-safe windows to model comparison

Day 3 promise

By the end of today, learners should be able to: - build a **leakage-safe windowing pipeline**, - compare **Dense AE**, **LSTM AE**, **forecast residuals**, and **Isolation Forest**, - choose a thresholding strategy that is operationally defensible, - and explain **why** a model fired, not just that it fired.

> This day is about **representation learning with discipline**, not about replacing every baseline with a neural network.

Agenda

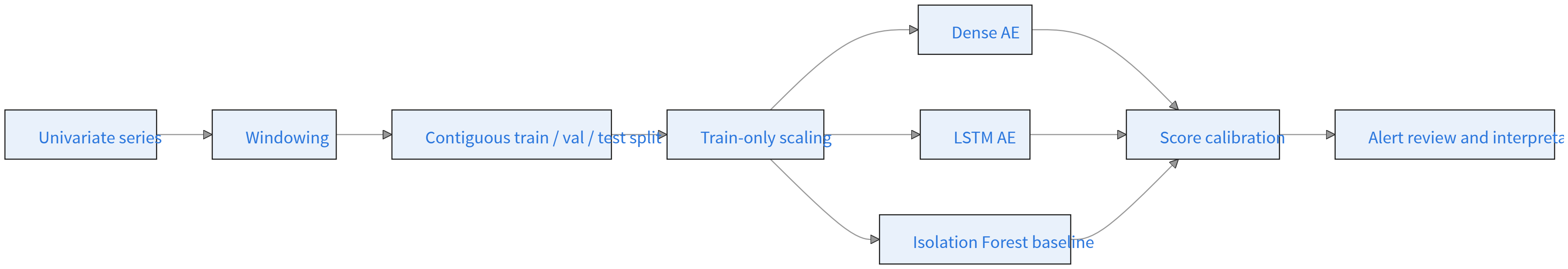
- failure types and why windows beat single timestamps,
- leakage-safe windowing and thresholding,
- Dense AE, LSTM AE, forecast residuals, Isolation Forest,
- thresholds, score interpretation, and the lab walkthrough.

Failure types we want learners to distinguish

- **Point anomaly** — one or a few sharp spikes
- **Collective anomaly** — a suspicious segment, even if no individual point is extreme
- **Drift anomaly** — behavior that slowly leaves the normal regime

> Professional teaching note: learners often think “anomaly” means “largest point.” Day 3 is where that misconception should be corrected.

Reference workflow



Leakage-safe discipline

1. Build windows first.
2. Split **in time order**.
3. Fit scaling on the **training slice only**.
4. Train reconstruction models on **normal or mostly-normal windows**.
5. Derive thresholds from the training distribution, not from the full series.

> If learners remember only one operational rule from Day 3, it should be this: **future information must not leak into scaling or thresholding**.

Model roles: what each branch is good for

Dense AE

- Fast baseline for fixed-length windows
- Good when shape is learnable without explicit recurrence
- Easier to debug than sequence models

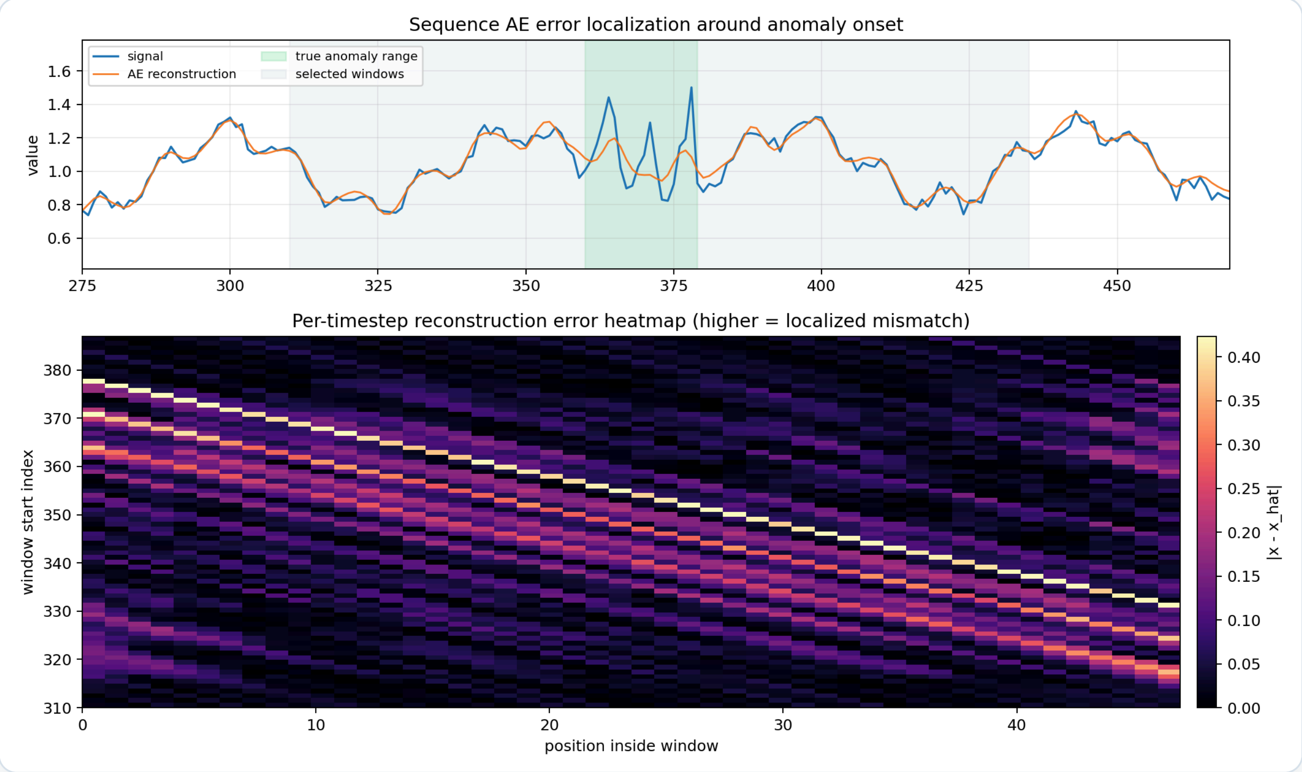
LSTM AE

- Better inductive bias for ordered sequences
- Often stronger on drift or smoother collective changes
- Produces **per-step reconstruction errors** that help localize an anomalous subsequence inside a window
- Slower and more sensitive to tuning

Forecast-residual detector

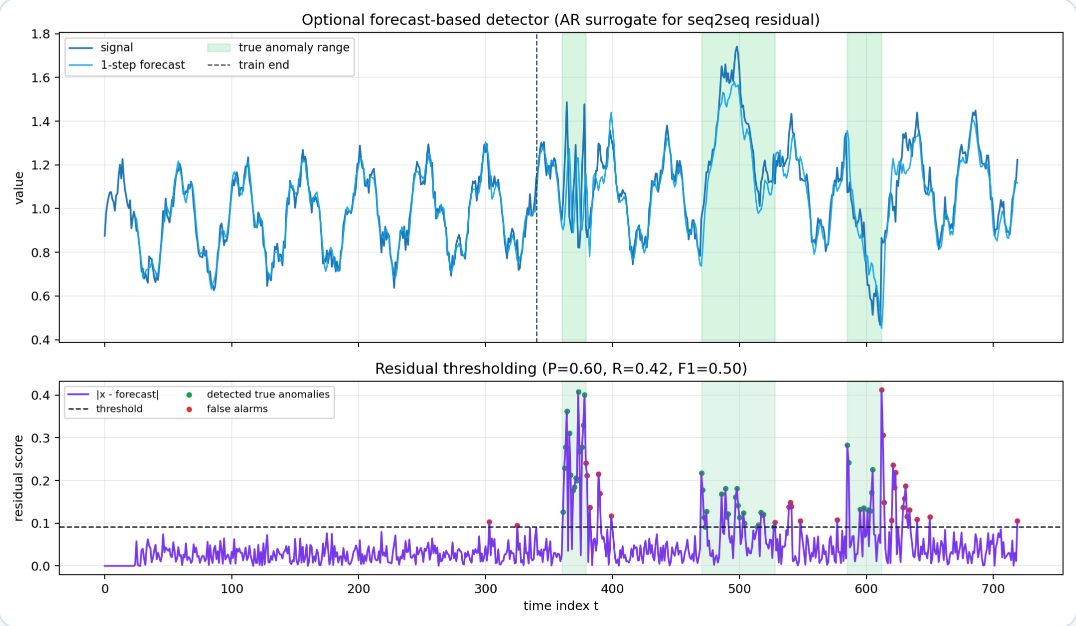
- Train a forecaster on normal behavior, score $|x_{t+1} - \hat{x_{t+1}}|$

Sequence-AE localization



- Dense AE returns one error per window; LSTM AE returns one error **per step**.
- Per-step errors highlight *where inside the window* the model was surprised.
- Use this when learners ask “the alert fired — but where did it actually go wrong?”

Forecast-residual path



- Score = $|x_{t+1} - \hat{x_{t+1}}|$ from a forecaster trained on normal history.
- Good for **point spikes** and **clean regime shifts** where the forecast baseline is stable.
- A useful third lens alongside reconstruction-based AEs and Isolation Forest — not a replacement.

Why Huber loss appears in the lab

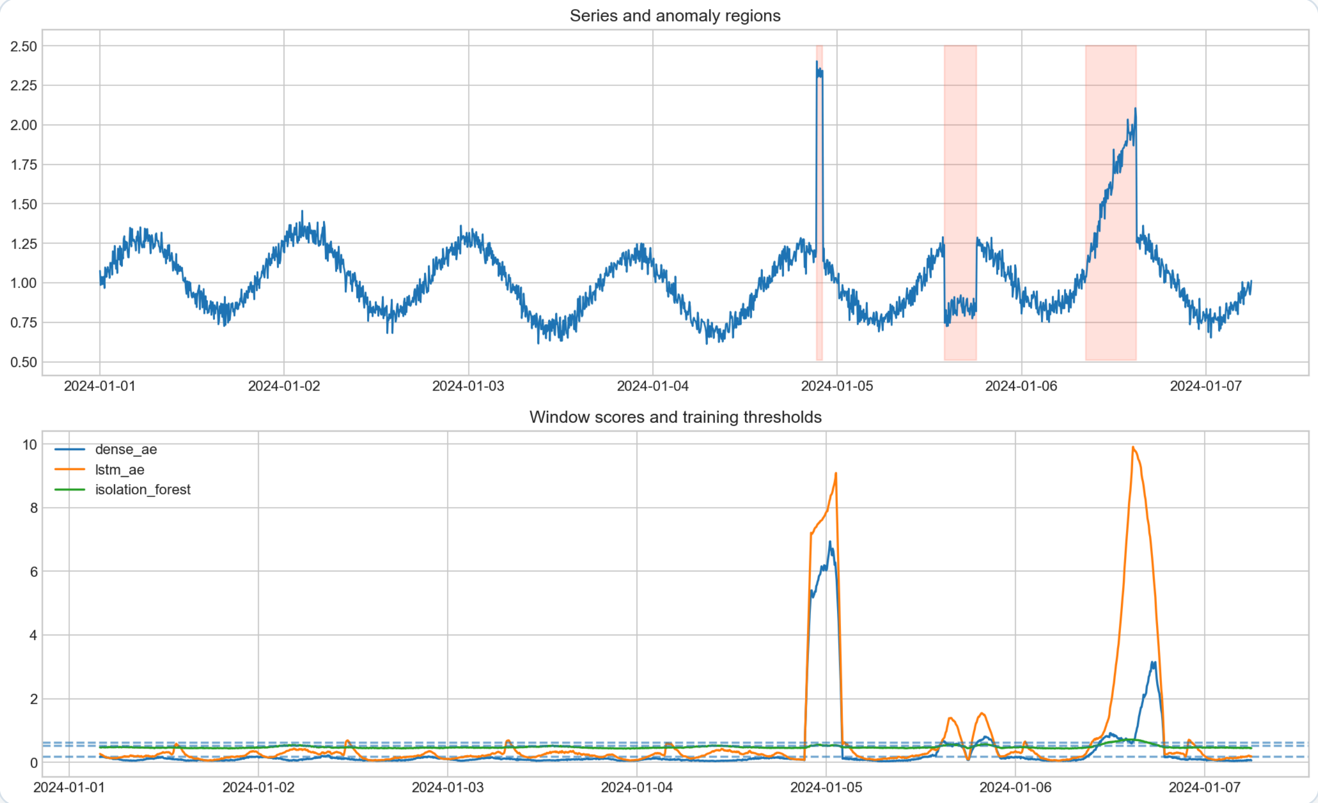
- MSE can be dominated by a few large reconstruction misses.
- Huber behaves like:
  - MSE near zero
  - L1 for larger errors
- In this training context, Huber is a good teaching choice because it is robust to **slightly contaminated training data**.

Worked teaching interpretation

If the LSTM AE fires but Isolation Forest does not, ask: - Did the anomaly change **shape over time** rather than one local point? - Did the window-level context matter more than a single spike?

If Isolation Forest fires first, ask: - Is this a sharp outlier that does not require temporal memory? - Did the AE over-smooth the behavior?

Demo result from the notebook



> On the default synthetic demo, the important classroom point is not only who wins. It is **why the ranking looks that way**.

What the lab actually showed

On the shipped synthetic seed (`day3_lab.ipynb`):

Model	Precision	Recall	F1
Dense AE	1.00	0.93	0.96
LSTM AE	0.94	0.87	0.90
Isolation Forest	1.00	0.72	0.84

Teaching reading: - Dense AE won on this seed — recall was the deciding axis. - Isolation Forest held perfect precision; the gap was missed anomalies, not false alarms. - LSTM AE was competitive, not dominant — a useful corrective against “sequence model is automatically better.”

How to teach the notebook

`../notebooks/day3_lab.ipynb`

Recommended facilitation checkpoints: - inspect the raw series before modeling, - explain why the split is contiguous, - compare the learned models against IF, - review score plots before discussing metrics, - end with "what still fails?"

Classroom discussion prompts

- When would you trust IF over the AEs?
- What happens when the training slice is contaminated?
- How would a different window length change what the model notices?
- What metric matters more in practice here: F1, top-k review quality, or false-alarm rate?

Common pitfalls

- treating neural models as automatically superior,
- fitting thresholds on the full dataset,
- discussing metrics without looking at score traces,
- ignoring the distinction between point and collective anomalies,
- not checking what the baseline already does well.

End-of-day learner deliverable

Each learner should leave with: - one clean notebook run, - one chart comparing model behavior, - one sentence on **when AE helped**, - one sentence on **when the baseline stayed competitive**, - one note on a remaining operational risk.